

Análise Computacional de Problemas de Escalonamento: Machine Scheduling, Job Shop e Flow Shop com JuMP/Julia

Bruna T. F. Ferreira¹

Aluna Especial

Relatório Técnico – Disciplina Otimização, PPGEE/UFAM

Abstract

Este trabalho apresenta uma análise computacional de três classes de problemas de escalonamento resolvidas com programação inteira mista (MILO) em Julia/JuMP e solver HiGHS: (1) *Machine Scheduling* $1|r_j|\sum T_j$ (31 instâncias, n de 5 a 24 jobs); (2) *Job Shop Scheduling* $J||C_{\max}$ (10 instâncias JSPLIB, até 20×10 jobs $\times$ máquinas); e (3) *Flow Shop Scheduling* $F|prmu|C_{\max}$ (7 instâncias CSV, até 30 jobs $\times$ 10 máquinas). Para o Machine Scheduling são comparadas três heurísticas (FIFO, EDD, SPT) ao MILO exato. Para o Flow Shop adiciona-se a heurística NEH. O limite de tempo foi fixado em 60 s por instância (configuração livre, conforme enunciado). Os resultados evidenciam a transição entre tratabilidade e intratabilidade computacional das três classes, e mostram que a NEH compete com o MILO exato em instâncias grandes de Flow Shop.

1. Ambiente Computacional

Hardware: AMD Ryzen 5 4500 (6 núcleos, 3,6 GHz), 16 GB RAM, Windows 11 x86_64-w64-mingw32.

Software: Julia v1.10.11, JuMP v1.30.1, HiGHS v1.14.0. O solver HiGHS foi utilizado em modo MILO com configuração padrão; única alteração: `time_limit = 60 s` e modo silencioso.

2. Modelos e Metodologia

2.1. Machine Scheduling ($1|r_j|\sum T_j$)

Uma única máquina processa n jobs. Cada job j tem liberação r_j , duração p_j e prazo d_j ; o objetivo é minimizar o **atraso total** $\sum_j \max(0, C_j - d_j)$. O modelo MILO usa a reformulação *big-M* por par:

$$C_i \leq s_j + Mz_{ij}, \quad C_j \leq s_i + M(1 - z_{ij}), \quad z_{ij} \in \{0, 1\}. \quad (1)$$

O número de binárias é $n(n-1)/2$. Três heurísticas de ordenação única servem de referência: FIFO (por r_j), EDD (por d_j) e SPT (por p_j).

Instâncias: 31 arquivos JSON – 30 instâncias geradas com esquema $1|r_j|\sum T_j$ ($n \in \{5, 7, 9, 11, 13, 15, 17, 19, 21, 24\}$, 3 seeds) mais `inst.book` ($n = 7$) do livro [1].

2.2. Job Shop Scheduling ($J||C_{\max}$)

n jobs em m máquinas, cada job com rota própria. Minimiza-se o makespan $C_{\max}$. As restrições de **precedência** (ordem dentro do job) e de **disjunção** por máquina (big-M com $H = \sum p_{jk}$) definem o modelo. **Instâncias JSPLIB [2]:** 10 benchmarks clássicos (ft06, la01–la04, ft10, ft20, abz5, abz6, orb01).

2.3. Flow Shop Scheduling ($F|prmu|C_{\max}$)

Todos os jobs percorrem as máquinas na **mesma ordem**. A **formulação posicional** (Wilson) usa $x_{jk} \in \{0, 1\}$ (job j na posição k) e recursos de conclusão *sem big-M*:

$$C_{k,i} \geq C_{k-1,i} + \bar{p}_{k,i}, \quad C_{k,i} \geq C_{k,i-1} + \bar{p}_{k,i}. \quad (2)$$

O número de binárias é n^2 . A heurística NEH [3] constrói a sequência inserindo cada job na posição que minimiza o makespan parcial. **Instâncias [4]:** 7 CSVs (3–10 máquinas, 10–30 jobs).

3. Resultados – Machine Scheduling

3.1. Escalabilidade

A Tabela 1 sumariza o tempo médio do MILO, a fração de instâncias resolvidas na otimalidade e o gap

Table 1: Machine Scheduling – resumo por n (3 inst./ponto, limite = 60 s)

n	$\bar{t}_{\text{MILO}}$ (s)	Fr. ót. (%)	$\bar{g}$ (%)	#bin
5	<0,1	100	—	10
7	1,1	100	—	21
9	2,6	100	—	36
11	27,8	67	73,6	55
13	42,1	33	50,9	78
15	52,5	33	89,5	105
17	60,0	0	91,2	136
19	60,0	0	96,8	171
21	60,0	0	96,2	210
24	60,0	0	99,0	276

$\bar{g}$: gap médio do solver (inst. TIME_LIMIT).

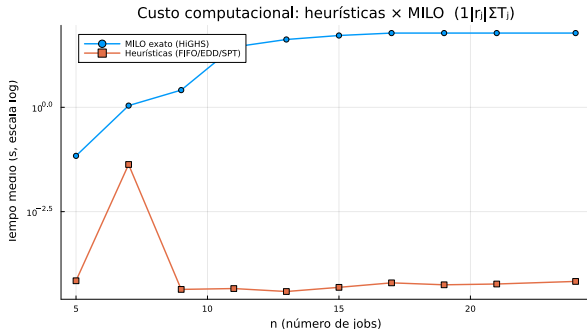


Figure 1: Tempo médio (escala log) vs. n – heurísticas são ordens de magnitude mais rápidas que o MILO.

médio quando o limite de tempo é atingido (média sobre as 3 instâncias de cada n).

O número de binárias cresce quadraticamente ($n(n-1)/2$), e o tempo de resolução explode a partir de $n = 11$: com apenas 55 binárias uma das três instâncias já atinge TIME_LIMIT. A partir de $n = 17$ nenhuma instância é resolvida na otimalidade dentro de 60 s; para $n = 24$, o gap médio do solver é de 99%, indicando que mal se encontra uma solução admissível razoável. A Fig. 1 ilustra a curva de complexidade.

3.2. Qualidade das heurísticas

Para $n = 5$, as três heurísticas encontraram o ótimo em todas as instâncias (excesso = 0%). Para $n \geq 7$, o excesso da melhor heurística sobre o ótimo oscilou entre 15% e mais de 100% dependendo da instância (Fig. 2), confirmando que rapidez de ordenação é paga em qualidade.

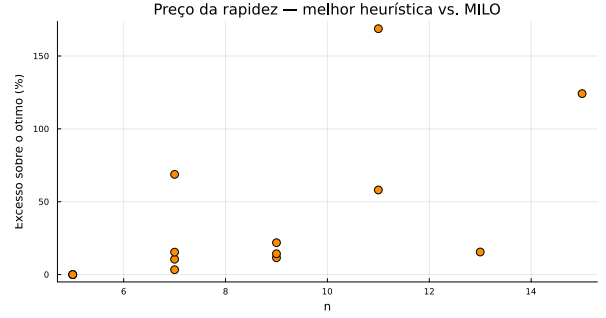


Figure 2: Excesso da melhor heurística sobre o ótimo MILO, para instâncias com status OPTIMAL.

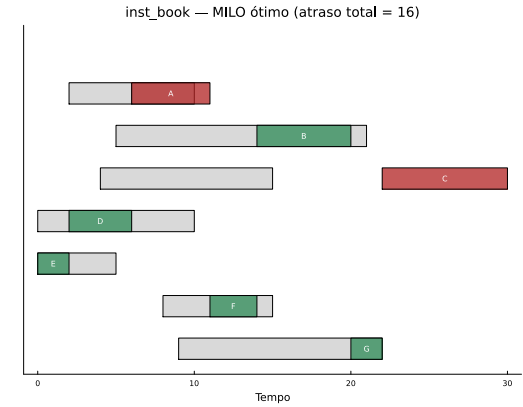


Figure 3: Gantt da inst_book – solução ótima ($\sum T_j = 16$).

3.3. Instância inst_book ($n = 7$)

O MILO encontrou o ótimo $\sum T_j^* = 16$ em 2,9 s. EDD obteve atraso 27 (excesso 68,8%), FIFO obteve 31 e SPT 51. O gráfico de Gantt (Fig. 3) mostra o escalonamento ótimo.

4. Resultados – Job Shop Scheduling

A Tabela 2 apresenta os resultados para as 10 instâncias JSPLIB. A coluna Δ é o gap em relação ao ótimo conhecido da literatura: $\Delta = 100 \cdot (\hat{C} - C^*)/C^*$. A coluna “gap MILO” é o gap interno do solver (distância da relaxação linear).

A única instância resolvida na otimalidade dentro de 60 s foi ft06 (6×6, 90 binárias, 1,2 s). Três instâncias 10×5 (la01, la02, la04) e abz6 (10×10) atingiram o valor ótimo da literatura mesmo sob TIME_LIMIT ($\Delta = 0$), mas o solver não conseguiu fechar o gap da relaxação linear dentro do tempo. Instâncias 10×10 com ordens tecnológicas complexas (ft10, orb01) apresentaram gaps $\Delta > 12\%$; ft20 (20×5) mostrou o maior desvio ($\Delta = 16,7\%$). A Fig. 4 exhibe o Gantt ótimo do

Table 2: Job Shop – MILO (HiGHS, 60 s) vs. ótimos da literatura

Inst.	$n \times m$	C^*	$\hat{C}$	Δ (%)	gap MILO(%)	St
ft06	6×6	55	55	0,0	0,0	OPT
la01	10×5	666	666	0,0	21,2	TL
la02	10×5	655	655	0,0	24,6	TL
la03	10×5	597	605	1,3	27,3	TI
la04	10×5	590	590	0,0	18,8	TI
abz5	10×10	1234	1268	2,8	22,9	TI
abz6	10×10	943	943	0,0	6,0	TL
ft10	10×10	930	1043	12,2	28,4	TL
ft20	20×5	1165	1360	16,7	65,8	TL
orb01	10×10	1059	1198	13,1	35,5	TL

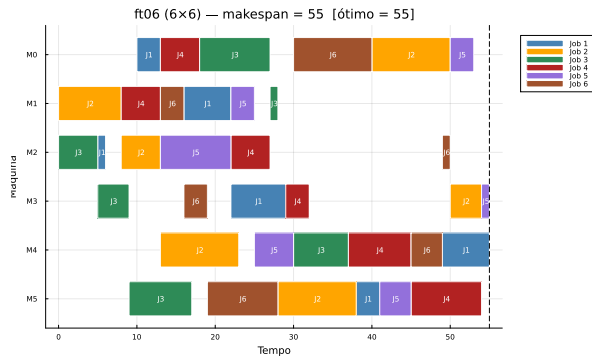
OPT: OPTIMAL; TL: TIME LIMIT. $\hat{C}$: melhor solução encontrada.

Figure 4: Gantt de ft06 – makespan ótimo = 55.

ft06, e a Fig. 5 compara obtido vs. ótimo para todas as instâncias.

5. Resultados – Flow Shop Scheduling

5.1. NEH vs. MILO posicional

A Tabela 3 apresenta os resultados completos. $\delta_{NEH} = 100 \cdot (NEH - MILO) / MILO$; valores negativos indicam que a NEH *superou* o MILO.

Seis das sete instâncias foram resolvidas na otimalidade (OPT) pelo MILO posicional dentro de 60 s. Na única instância que atingiu TIME_LIMIT (10m_20j, $n^2 = 400$ binárias), a NEH produziu makespan 292 versus 300 do MILO – a **heurística superou o exato** dentro do orçamento de tempo. A Fig. 6 exibe o Gantt da 3m_10j via NEH, e a Fig. 7 compara os dois métodos em todas as instâncias.

6. Discussão e Conclusões

Machine Scheduling. O MILO big-M resolve instâncias até $n = 9$ com facilidade (<3 s). Entre $n = 11$

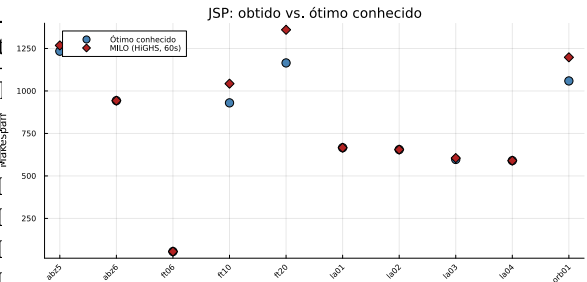


Figure 5: JSP: makespan obtido pelo MILO vs. ótimo da literatura.

Table 3: Flow Shop – NEH vs. MILO posicional (HiGHS, 60 s)

Inst.	m	n	NEH	MILO	δ_{NEH} (%)	St.
3m_10j	3	10	126	126	0,0	OPT
5m_10j	5	10	132	129	+2,3	OPT
8m_10j	8	10	197	187	+5,3	OPT
3m_20j	3	20	218	218	0,0	OPT
5m_20j	5	20	269	263	+2,3	OPT
5m_30j	5	30	327	327	0,0	OPT
10m_20j	10	20	292	300	-2,7	TL

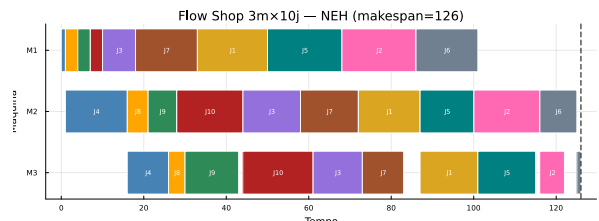
 $\delta < 0$: NEH superou o MILO (que atingiu TL).

Figure 6: Gantt da 3m_10j – sequência NEH (makespan = 126).

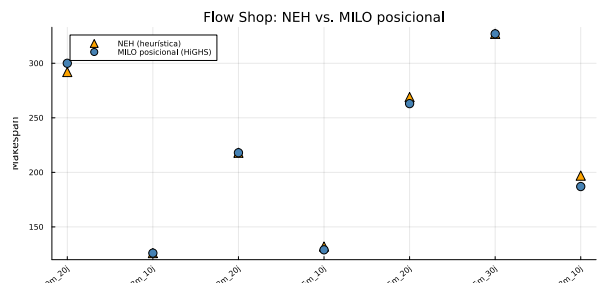


Figure 7: Flow Shop: NEH vs. MILO posicional. Na instância 10m_20j a NEH superou o MILO (TIME_LIMIT).

e $n = 15$ há uma *transição*: algumas instâncias são ótimas, outras atingem TIME.LIMIT – dependendo da dificuldade particular da instância. A partir de $n = 17$, o modelo (136 binárias) é sistematicamente intratável em 60 s. As heurísticas, por sua vez, são prática zero em tempo, mas o excesso sobre o ótimo pode ultrapassar 100% – confirmando que rapidez tem custo em qualidade.

Job Shop. Apenas ft06 (6×6) foi resolvido na otimalidade. Os 450 binárias das instâncias 10×10 inviabilizam a resolução exata em 60 s; ainda assim, em abz6 e nas 10×5 (exceto la03) o solver encontrou soluções de valor ótimo sem conseguir provar a otimalidade. ft20 (20×5) foi surpreendentemente difícil (950 binárias, $\Delta = 16,7\%$), evidenciando que o número de jobs domina a complexidade mesmo com poucas máquinas.

Flow Shop. A formulação posicional (sem big-M) resolve 6 de 7 instâncias na otimalidade. O caso 10m_20j ilustra o ponto de transição: 400 binárias com acoplamentos em 10 máquinas × 20 posições esgotam o tempo, e a NEH – barata e de alta qualidade – supera o MILO dentro do orçamento. Isso reforça o valor de heurísticas construtivas fortes como alternativa (ou ponto de partida/*warm-start*) ao exato em instâncias de médio/grande porte.

Em todos os três problemas, os resultados confirmam o princípio fundamental: **modelos MILO escalam mal para problemas NP-difíceis** – a exploração da árvore branch-and-bound cresce exponencialmente com o número de binárias. A configuração livre do HiGHS (gap, cortes, heurísticas internas) pode melhorar o desempenho em instâncias específicas, mas não elimina o problema estrutural de escalabilidade.

References

- [1] C. Guéret, C. Prins, M. Sevaux, *Applications of Optimization with Xpress-MP*. Dash Optimization, 2000.
- [2] T. Yamada, R. Nakano, “A Genetic Algorithm Applicable to Large-Scale Job-Shop Problems,” *Parallel Problem Solving from Nature 2*, 1992. Repositório: <https://github.com/tamy0612/JSPLIB>.
- [3] M. Nawaz, E. E. Enscore, I. Ham, “A Heuristic Algorithm for the m -Machine, n -Job Flow-Shop Sequencing Problem,” *Omega*, 11(1):91–95, 1983.
- [4] A. Elkamel, A. Gharbi, Flow Shop Scheduling Problem Dataset. <https://github.com/akilelkamel/fssp-dataset>, 2021.